Prompt 6

KKC TEXT ADVENTURE — PROMPT 6 OF 180
==========
Free-Tier LLM Narration Adapter (Gemini First, Local Fallback Required)
==========

This prompt builds directly on Prompts 1, 2, 3, 4, and 5.
Do not restructure, rename, or refactor anything from those prompts.
Only add to what exists.

This prompt returns to the original Phase 1 plan role for Prompt 6:
adding a narration layer. However, unlike the original plan, this
must use a free-tier-capable API and must remain optional. The game
must still run locally without network access when the API key is absent.

==========
HARD BANS — UPDATED FOR THIS PROMPT
==========

You may add ONE optional LLM narration provider using the
Google Gemini Developer API.

Do not add:
- Anthropic / Claude SDK
- OpenAI SDK
- axios, fetch wrappers, or extra HTTP libraries beyond what the
  chosen SDK requires
- any web server or frontend
- any architecture that moves game truth into the LLM
- any system where the game stops working if the LLM is unavailable

The engine remains the source of truth.
The LLM is prose only.

==========
ARCHITECTURE RULE — STILL ABSOLUTE
==========

engine/* owns:
- command parsing
- movement
- access checks
- time advancement
- state mutation
- persistence
- sympathy truth
- social consequence truth

- music/Eolian truth

narration/* owns:
- prose rendering only

The LLM provider:
- must never mutate PlayerState
- must never write to the database
- must never invent new exits, items, NPCs, locations, rules, or lore
- must never resolve mysteries
- must only narrate from structured context passed in by the engine

repl.ts still saves state exactly once per turn.
Do not reload state from DB after narration.

═══════════════════════════════════════════════════════
GOAL OF THIS PROMPT
═══════════════════════════════════════════════════════

By the end of Prompt 6, the project should support two narration modes:

1. Local deterministic narration (existing system) — always works
2. Optional Gemini narration — used only when GEMINI_API_KEY is set

The player should be able to:
- run the game with no API key and still get the current local narration
- set GEMINI_API_KEY and receive prose narration from Gemini for supported commands
- fall back automatically to local narration if the Gemini call fails
- preserve all engine truth and state behaviour from Prompts 1–5
- keep narration canon-constrained and mystery-safe

═══════════════════════════════════════════════════════
SCOPE OF THIS PROMPT
═══════════════════════════════════════════════════════

This prompt covers:
1. Add an optional Gemini-based narration provider
2. Extend the NarrationProvider interface only as needed
3. Add a narration context builder
4. Add a strict system instruction / canon guard for Gemini
5. Add runtime selection: local only if no API key, Gemini if present
6. Add graceful fallback to local narration on any Gemini failure
7. Add tests for fallback behaviour and prompt construction

This prompt does NOT cover:
- OpenAI or Anthropic integration
- NPC freeform conversation via LLM

- allowing the LLM to decide outcomes
- mystery resolution
- Denna, Devi, Auri, Bast, or other fragile NPCs
- web deployment
- prompt streaming
- chat history or long memory

═══════════════════════════════════════════════════════════

NEW FILES IN THIS PROMPT

═══════════════════════════════════════════════════════════

Add exactly these files:

```
src/
  narration/
    geminiNarrator.ts        ← optional Gemini provider
    narrationContext.ts      ← build structured narration context
    canonInstructions.ts     ← strict narration instructions
tests/
  geminiNarrator.test.ts
```

Modify these existing files:

- src/types/index.ts
- src/narration/provider.ts
- src/narration/localNarrator.ts
- src/engine/actions.ts
- src/repl.ts
- src/index.ts
- package.json
- .env.example

Do not add any other files.

═══════════════════════════════════════════════════════════

DEPENDENCIES

═══════════════════════════════════════════════════════════

Add exactly one new dependency:

dependencies:
  @google/genai

Do not add any other new packages.

Keep all existing packages unchanged.

═══════════════════════════════════════════════════════════

ENVIRONMENT VARIABLES

═══════════════════════════════════════════════

Update .env.example to include:

```
DB_PATH=./data/game.db
NODE_ENV=development
GEMINI_API_KEY=
NARRATION_MODE=auto
```

Rules:
- GEMINI_API_KEY may be blank
- NARRATION_MODE may be:
  auto
  local
  gemini

Behaviour:
- local  → always use LocalNarrator
- gemini → try GeminiNarrator, but still fall back to local on error
- auto   → use GeminiNarrator if GEMINI_API_KEY is set, otherwise local

Do not add any other env vars.

═══════════════════════════════════════════════
TYPES — additions to src/types/index.ts
═══════════════════════════════════════════════

Do not remove or change existing types unless explicitly required.

Add:

```
export interface NarrationSceneContext {
  command: string;
  player_summary: {
    era: string;
    location_id: string;
    time_of_day: TimeOfDay;
    day_number: number;
    money_drabs: number;
    academic_rank: AcademicRank;
    hunger: number;
    fatigue: number;
    warmth?: number;
    injuries: string[];
  };
  location_summary: {
    id: string;
    name: string;
```

```
    description_base: string;
    exits: string[];          // directions only
  };
  npc_summary: {
    names: string[];
  };
  engine_truth: {
    movement_message?: string;
    sympathy_outcome?: string;
    social_outcome?: string;
    music_outcome?: string;
  };
}
```

Update NarrationProvider so it supports one additional method:

```
  narrateScene?(context: NarrationSceneContext): Promise<string>;
```

Rules:
- This method is optional on the interface
- LocalNarrator does not need to implement it
- GeminiNarrator must implement it

Do not change existing renderLocation/renderWait/renderFallback/renderHelp methods.

═══════════════════════════════════════════════════
NARRATION CONTEXT — src/narration/narrationContext.ts
═══════════════════════════════════════════════════

Implement:

```
buildNarrationSceneContext(
  command: string,
  state: PlayerState,
  location: Location,
  npcs: NPC[],
  accessibleExits: Exit[],
  engineTruth?: {
    movement_message?: string;
    sympathy_outcome?: string;
    social_outcome?: string;
    music_outcome?: string;
  }
): NarrationSceneContext
```

Rules:
- exits must be directions only

- include only currently accessible exits
- do not include any hidden or blocked exits
- do not include any unresolved lore facts
- do not include raw trust numbers or hidden flags
- keep the context compact and deterministic

This file is pure. It does not call the database.

═══════════════════════════════════════════════════════════

CANON INSTRUCTIONS — src/narration/canonInstructions.ts

═══════════════════════════════════════════════════════════

Export:

KKC_CANON_INSTRUCTIONS: string

This is a strict instruction block for the Gemini narrator.

It must include all of the following rules:

- You are narrating a Kingkiller Chronicle scene.
- You are not the source of lore truth.
- The structured context is authoritative.
- Do not invent locations, exits, NPCs, objects, or rules.
- Do not resolve mysteries.
- Never state the true nature of the Chandrian, Amyr, Denna's patron, Doors of Stone, or hidden names.
- Do not add modern idiom.
- Use present tense.
- Use second person.
- Keep prose to 2–4 short paragraphs maximum.
- Make the scene grounded, materially specific, and restrained.
- If engine_truth contains a movement, sympathy, social, or music outcome, honour it exactly. Do not contradict it.
- If the command is simple observation, narrate only what is supported by the context.
- If uncertain, be quieter rather than inventing.

Do not quote Rothfuss.
Do not include any raw JSON in the output.

═══════════════════════════════════════════════════════════

GEMINI NARRATOR — src/narration/geminiNarrator.ts

═══════════════════════════════════════════════════════════

Implement GeminiNarrator as a class.

Constructor:

new GeminiNarrator(localFallback: LocalNarrator)

Rules:
- It must use @google/genai
- It must read GEMINI_API_KEY from process.env
- If GEMINI_API_KEY is missing, it must throw a clear constructor error
- It must not write to the DB
- It must not mutate PlayerState
- It must use LocalNarrator as a fallback path

Implement:

narrateScene(context: NarrationSceneContext): Promise<string>

Behaviour:
- Build a request from:
  - KKC_CANON_INSTRUCTIONS
  - a compact structured summary of the context
- Use a free-tier-capable Gemini model.
- Default model string:
  'gemini-2.5-flash-lite'
- Keep output modest in length
- On any error, invalid output, or empty text:
  return a locally rendered fallback scene string

Also implement a private helper:
validateOutput(text: string): boolean

Validation rules:
- reject empty text
- reject text over 1600 characters
- reject text containing any of these case-insensitive phrases:
  "Chandrian are"
  "the Amyr are"
  "Denna's patron is"
  "the Doors of Stone are"
  "true name of"
- reject text containing obvious modern slang:
  "okay", "cool", "awesome", "weird vibe", "NPC", "game"

If validation fails, use the local fallback.

The local fallback should be:
- for scene narration, LocalNarrator.renderLocation(...)
- for unsupported contexts, a short existing local fallback line

Do not add streaming.
Do not add conversation memory.

═══════════════════════════════════════════════

NARRATION PROVIDER — update src/narration/provider.ts

═══════════════════════════════════════════════

Keep the existing re-export.
Also export any new narration-related types if needed by the codebase.

Do not add logic here.

═══════════════════════════════════════════════

LOCAL NARRATOR — update src/narration/localNarrator.ts

═══════════════════════════════════════════════

Do not refactor existing local narration behaviour.
Only add one helper method if needed:

```
renderSceneFromContext(context: NarrationSceneContext): string
```

This should produce a local deterministic scene string using existing
renderLocation() plus any engine_truth line prepended if present.

Rules:
- Keep it simple
- Reuse existing rendering logic
- Do not mutate anything
- This helper exists mainly so GeminiNarrator has a clean fallback path

If you can implement Gemini fallback without adding this helper, do that instead.
Do not add unnecessary abstraction.

═══════════════════════════════════════════════

ACTIONS — update src/engine/actions.ts

═══════════════════════════════════════════════

Do not refactor working command logic.
Only extend output handling where needed.

For commands that currently return:
- movement + location rendering
- wait + location rendering
- sympathy outcome
- social outcome
- Eolian outcome

Add the ability to produce a NarrationSceneContext and call
narrator.narrateScene(context) if:
- narrator has narrateScene defined

- and NARRATION_MODE is not 'local'

Rules:
- Engine still decides the newState
- Engine still decides all outcome truth
- LLM narration is only the prose wrapper
- If narrateScene is unavailable, use existing local rendering path
- help, status, inventory, and quit remain local only
- talkToNPC remains local only in this prompt

Supported Gemini scene commands in this prompt:
- look / look around
- go [direction] / bare direction
- wait
- use sympathy / bind ...
- ask kilvin for work
- work fishery
- ignore ambrose
- answer ambrose carefully
- answer ambrose sharply
- audition for pipes
- play at eolian

Do not change command semantics.
Do not add new commands.

═══════════════════════════════════════════════════════
REPL / STARTUP — update src/repl.ts and src/index.ts
═══════════════════════════════════════════════════════

In index.ts:
- import 'dotenv/config' remains first
- instantiate LocalNarrator always
- instantiate GeminiNarrator only if mode requires it and GEMINI_API_KEY exists
- choose the narrator at startup using:
  NARRATION_MODE and GEMINI_API_KEY

Selection rules:
- if NARRATION_MODE === 'local': use LocalNarrator
- else if NARRATION_MODE === 'gemini':
  try GeminiNarrator, fall back to LocalNarrator on constructor error
- else if NARRATION_MODE === 'auto':
  use GeminiNarrator if GEMINI_API_KEY exists, otherwise LocalNarrator

In repl.ts:
- do not change persistence behaviour
- do not change save timing
- no other startup complexity

## PACKAGE.JSON

Keep all existing scripts.
Add no new scripts unless required for test execution.

No web tooling.
No bundlers.
No extra config files.

## TESTS — tests/geminiNarrator.test.ts

Do not make real network calls in tests.

Mock the Gemini client.

Tests:

buildNarrationSceneContext:
1. produces directions-only exits
2. excludes blocked exits
3. includes engine_truth fields when provided

GeminiNarrator.validateOutput:
4. accepts short normal text
5. rejects forbidden mystery phrase
6. rejects modern slang phrase
7. rejects overly long output

GeminiNarrator fallback:
8. missing API key constructor throws clear error
9. API failure returns local fallback text
10. invalid Gemini output returns local fallback text

startup selection:
11. NARRATION_MODE=local uses LocalNarrator
12. NARRATION_MODE=auto without GEMINI_API_KEY uses LocalNarrator

Add 1 integration-style test to an existing actions-related test file:
13. when narrateScene exists and returns text, dispatch() still returns unchanged engine truth in newState and only swaps the prose output

## SUCCESS CRITERIA

==========

This prompt is complete when:

- the game still runs fully offline with no API key
- setting NARRATION_MODE=local preserves existing behaviour
- setting GEMINI_API_KEY and NARRATION_MODE=auto enables Gemini prose
- if Gemini fails, local narration is used automatically
- help, status, inventory, quit remain local-only
- talkToNPC remains local-only in this prompt
- the engine still owns all truth and state changes
- the Gemini layer never mutates PlayerState or DB state
- forbidden mystery resolution phrases are rejected
- all new tests pass alongside earlier prompts' tests
- no refactoring of working Prompts 1–5 code beyond what is required

==========
END OF PROMPT 6
==========